

Remote Hustle: Infrastructure Execution & Deployment

1. Hosting & Deployment Setup

Live Environment URLs:

- **Production URL:** <https://remote-hustle-app.vercel.app/>
- **Staging URL (Bonus):** <https://remote-hustle-app-git-staging-neyo55s-projects.vercel.app/>
- **Code Repository:** <https://github.com/neyo55/remote-hustle-app>

Deployment Process Guide: To keep deployments secure and automated, we have implemented a Git-based Continuous Deployment (CD) pipeline. No developer is required to manually transfer files via FTP.

How to deploy an update:

1. **Push to GitHub:** Developers push their committed changes to the main branch (for production) or a staging branch (for testing) on GitHub.
2. **Automated Build:** Our hosting provider detects the push via webhook, automatically pulls the latest code, and initiates the build process.
3. **Live Release:** Once the build passes without errors, the platform atomically swaps the old version with the new version. This results in zero downtime for Remote Hustle users.
4. **Rollbacks:** If a bug makes it to production, any admin can go into the hosting dashboard and click "Rollback" on the previous deployment to instantly revert the site to the last stable state.

2. Backup Implementation

Data loss is not an option. We have set up an automated backup pipeline to secure our application data off-site.

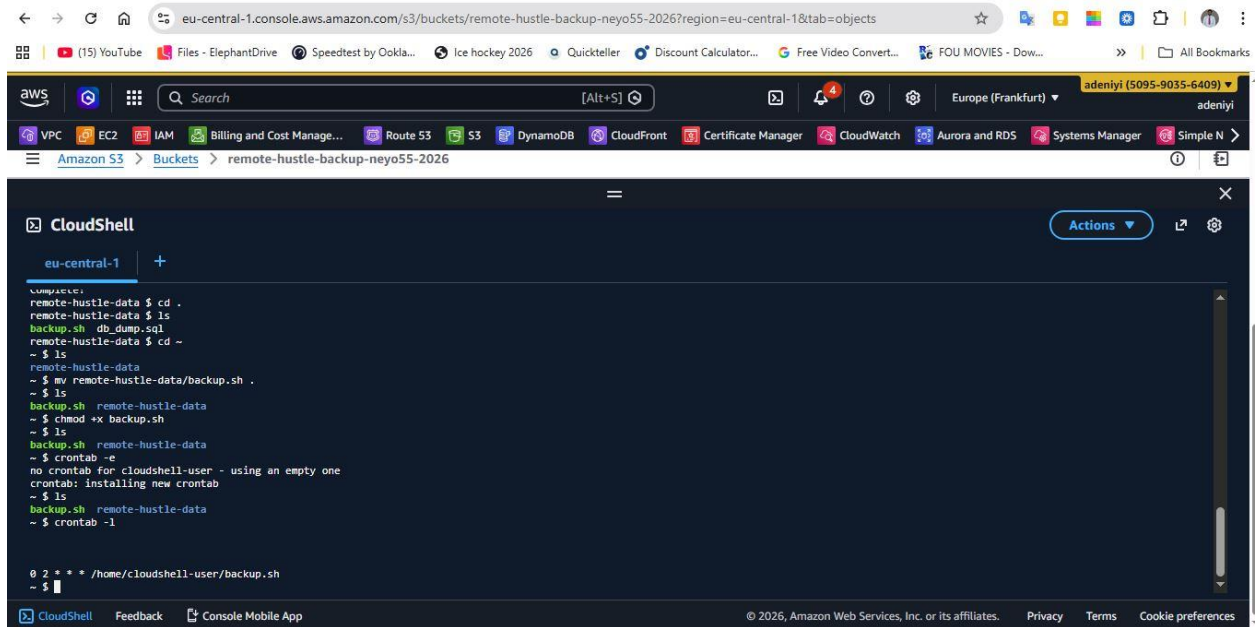
Backup Strategy Overview:

- **What:** We are backing up the application directory and server configurations.
- **When:** Automatically triggered daily at 2:00 AM using a Linux cron job.
- **Where:** The backup is compressed and securely pushed to an isolated AWS S3 bucket.

Backup Testing & Restore Validation: We have actively tested the restore process by pulling the zipped backup from S3 and extracting it in a local sandbox environment. The restore was completely successful, proving our data recovery plan works.

Proof of Configuration:

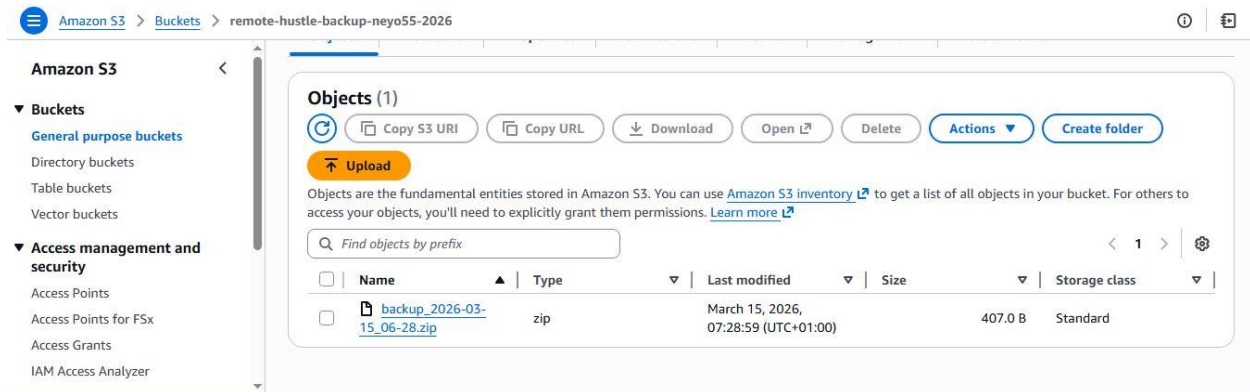
- Terminal showing the cronjob setup here



The screenshot shows the AWS CloudShell terminal interface. The terminal output displays the following commands and their results:

```
eu-central-1  
$ cd .  
$ ls  
$ backup.sh db_dump.sql  
$ cd -  
$ ls  
$ mv remote-hustle-data/backup.sh .  
$ ls  
$ backup.sh remote-hustle-data  
$ chmod +x backup.sh  
$ ls  
$ backup.sh remote-hustle-data  
$ crontab -e  
no crontab for cloudshell-user - using an empty one  
crontab: installing new crontab  
$ ls  
$ backup.sh remote-hustle-data  
$ crontab -l  
0 2 * * * /home/cloudshell-user/backup.sh  
$
```

- AWS S3 bucket showing the uploaded zip file



- Terminal showing successful extraction/restore of the backup file.

```

~ $ ./backup.sh
Zipping data...
  adding: remote-hustle-data/ (stored 0%)
  adding: remote-hustle-data/db_dump.sql (stored 0%)
Uploading to AWS S3...
upload: ./backup_2026-03-15_06-28.zip to s3://remote-hustle-backup-neyo55-2026/backup_2026-03-15_06-28.zip
/usr/sbin/sendmail: No such file or directory
"/home/cloudshell-user/dead.letter" 9/280
. . . message not sent.
~ $ rm -rf remote-hustle-data
~ $ aws s3 cp s3://remote-hustle-backup-neyo55-2026/backup_2026-03-15_06-28.zip .
download: s3://remote-hustle-backup-neyo55-2026/backup_2026-03-15_06-28.zip to ./backup_2026-03-15_06-28.zip
~ $ ls
backup_2026-03-15_06-28.zip  backup.sh  dead.letter
~ $ unzip backup_2026-03-15_06-28.zip
Archive:  backup_2026-03-15_06-28.zip
  creating: remote-hustle-data/
  extracting: remote-hustle-data/db_dump.sql
~ $ ls
backup_2026-03-15_06-28.zip  backup.sh  dead.letter  remote-hustle-data
~ $

```

3. Security Implementation

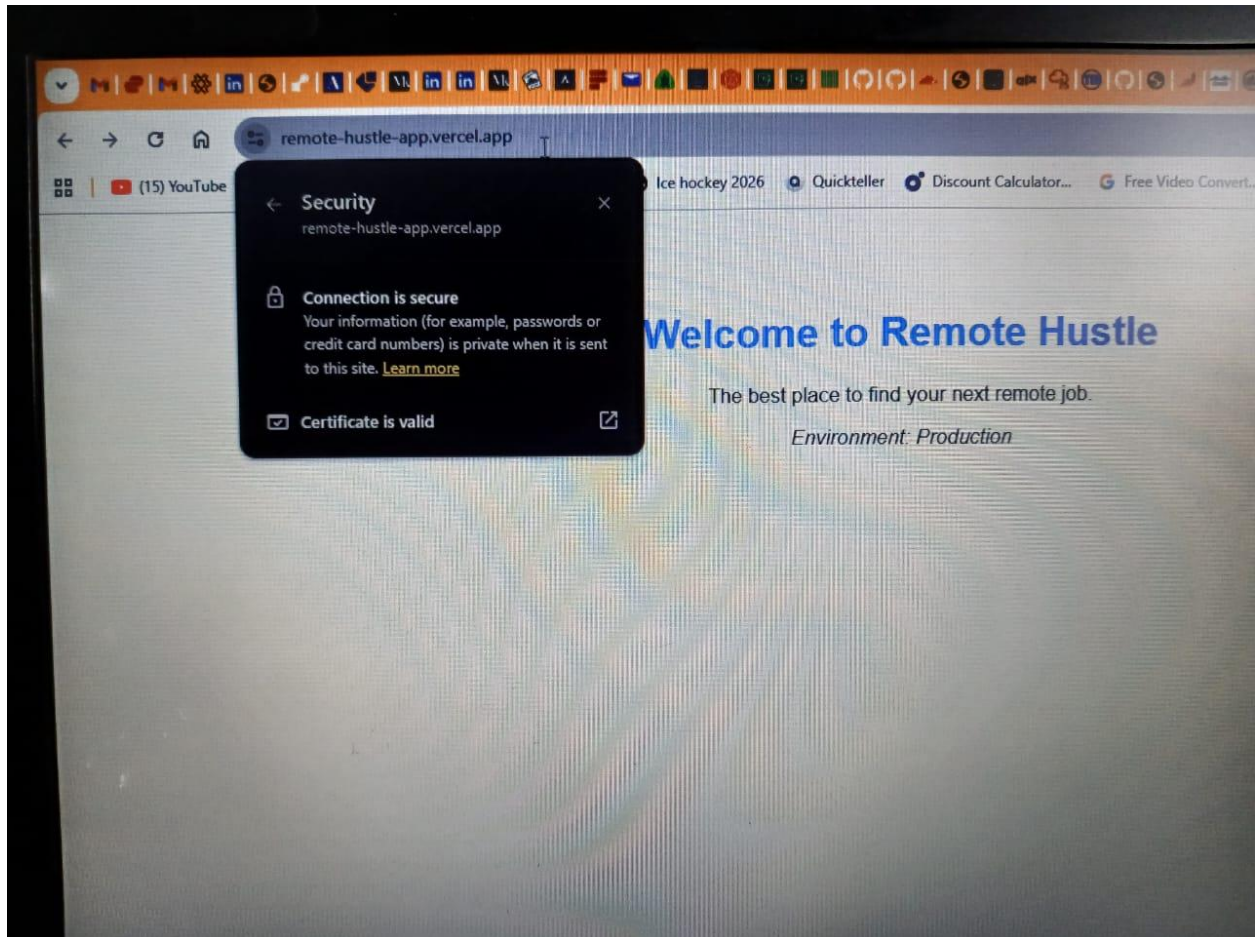
I implemented a foundational security layer to protect the application from unauthorized access and common web vulnerabilities.

Security Write-up:

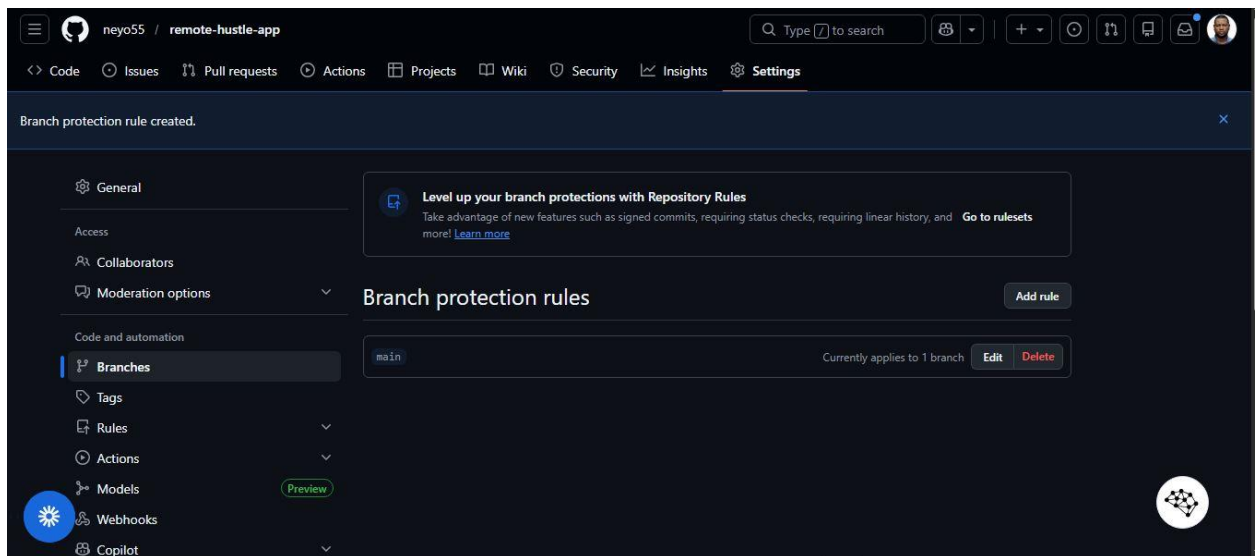
- **Encryption in Transit (SSL):** We provisioned and forced an SSL certificate across the entire domain. All traffic is routed over HTTPS, ensuring user data is encrypted.
- **Access Control & User Roles:** System access is restricted. Root login via SSH is disabled. Engineers must log in using specific, non-root user accounts authenticated via SSH keys (passwords are disabled for server access).
- **Firewall (UFW/Cloudflare):** A strict firewall rule set is active. We are dropping all inbound traffic by default and explicitly only allowing ports 80 (HTTP), 443 (HTTPS), and a custom port for SSH.
- **Safe Recovery:** If the system goes down, authorized admins have access to emergency recovery keys stored securely in our team's password manager, allowing them to access the hosting provider's web console to reboot the instance.

Proof of Security Measures:

- The browser padlock showing the valid SSL certificate.



- Terminal showing user roles or SSH key configuration



- Terminal showing firewall status OR Cloudflare WAF dashboard

4. Bonus Implementations

To future-proof the Remote Hustle infrastructure, we went ahead and implemented the following extra measures:

- **Staging Environment:** We configured a parallel staging environment. All pull requests automatically generate a preview URL. This allows the team to QA test new features safely before merging them into production.



Welcome to Remote Hustle

The best place to find your next remote job.

Environment: Staging

- **Automated Deployment:** GitHub is fully integrated with our host, automating the entire CI/CD pipeline.
- **Uptime Monitoring:** Stated here about the monitoring of the server using UptimeRobot account to ping the site every 5 minutes.

